// DEEP DIVE

VLANs & Trunking

FIG_000 · guide v1.0 · 2026 · open reference · 9 sections

by Swastik Sagar

How one physical switch fabric splits into many isolated logical networks — 802.1Q tagging, access vs trunk ports, inter-VLAN routing, and the security pitfalls that catch people off guard.

```
$ read section_01 --start
```

WHAT'S INSIDE

SECTION	TOPIC
1 — 2	What VLANs are, why they exist, and how 802.1Q tagging works
3 — 4	Access vs trunk ports, and a full frame-tagging walkthrough
5 — 6	Inter-VLAN routing and VLAN Trunking Protocol (VTP)
7 — 9	Native VLAN pitfalls, VLAN security, and a worked configuration example

How to use this guide: read start to finish for the full picture, or jump to section 4 if you just need the tagged-frame walkthrough. Every diagram is numbered and referenced directly in the surrounding text.

// INDEX

Table of Contents

This guide is designed to be read section by section, with diagrams positioned next to the concepts they illustrate.

// SECTION 1

What Are VLANs & Why They Exist

1.1 THE PROBLEM: ONE SWITCH, MANY NETWORKS

A **VLAN (Virtual Local Area Network)** splits a single physical switch — or an entire fabric of interconnected switches — into multiple logically isolated broadcast domains, as if each VLAN were its own separate physical network, without needing separate physical switches for each one. Standardized in **IEEE 802.1Q**, VLANs are one of the most fundamental tools in enterprise network design.

Before VLANs, isolating traffic between, say, an accounting department and an engineering department meant physically separate switches and cabling for each. That doesn't scale — every new department or security zone would mean buying and wiring up entirely new hardware. VLANs solve this by letting a single switch carry multiple logically-separated networks over the same physical infrastructure.

> **Broadcast domain:** the set of devices that receive each other's broadcast traffic (like ARP requests). Normally, everything on one physical switch is one broadcast domain. VLANs split that single switch into several independent broadcast domains — traffic in VLAN 10 never reaches a device in VLAN 20, even though both are plugged into the exact same physical switch.

1.2 WHY SEGMENT A NETWORK AT ALL

- **Security** — isolate sensitive systems (finance, HR, security cameras) from general user traffic, so a compromised device in one VLAN can't directly reach another.
- **Performance** — smaller broadcast domains mean less broadcast traffic flooding every device; a broadcast storm in one VLAN doesn't affect the others.
- **Organizational clarity** — group devices logically (by department, function, or floor) instead of being constrained by physical wiring layout.
- **Cost efficiency** — one physical switch can now serve the role of many, cutting hardware, cabling, and rack space needs dramatically.

1.3 VLANS ARE LOGICAL, NOT PHYSICAL

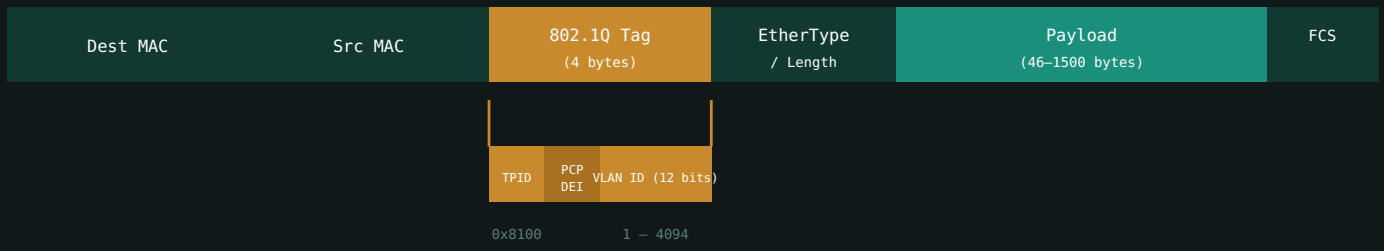
Two devices plugged into ports right next to each other on the same physical switch can be in completely different VLANs and unable to talk to each other directly at all — while a device three switches away, in a different building, can be in the *same* VLAN and communicate as if it were on the same local wire. Physical proximity stops mattering; VLAN membership is what defines the network boundary.

// SECTION 2

How VLAN Tagging Works (802.1Q)

2.1 THE 802.1Q TAG

For a switch to know which VLAN a frame belongs to as it crosses a link shared by multiple VLANs, the frame needs to carry that information somehow. **802.1Q** solves this by inserting a 4-byte **tag** into the Ethernet frame header, right after the source MAC address and before the EtherType field. This tag includes a 12-bit **VLAN ID** field, allowing up to 4,094 usable VLANs (0 and 4095 are reserved).



TPID (0x8100) marks this as a tagged frame; the 12-bit VLAN ID identifies which VLAN it belongs to

Fig 2.1 — The 802.1Q tag inserted into an Ethernet frame: a fixed TPID marker, priority bits, and the 12-bit VLAN ID.

2.2 TAGGED VS UNTAGGED FRAMES

A frame is either **tagged** (carries the 802.1Q header, explicitly marked with a VLAN ID) or **untagged** (no tag at all, implicitly belonging to whatever VLAN the receiving port is configured for). End devices — laptops, printers, phones — almost never send tagged frames themselves; they simply send normal Ethernet frames and rely on the switch port to know which VLAN they belong to. Tags matter specifically on links that carry *multiple* VLANs at once, which is exactly what Section 3 covers.

> **Key insight:** VLAN tagging is a switch-to-switch (or switch-to-router) concern. A typical end-user device has no idea VLANs exist — it just sends ordinary frames, and the network infrastructure handles the tagging transparently.

// SECTION 3

Access Ports vs Trunk Ports

3.1 ACCESS PORTS

An **access port** belongs to exactly one VLAN and connects to a single end device — a PC, printer, or IP phone. Frames arriving from the device are untagged; the switch internally associates them with the port's configured VLAN. Frames leaving toward the device are also sent untagged, since the device itself doesn't understand 802.1Q tags. From the end device's perspective, VLANs are completely invisible.

3.2 TRUNK PORTS

A **trunk port** carries traffic for *multiple* VLANs over a single physical link, typically used between switches, or between a switch and a router. Every frame crossing a trunk is tagged with its VLAN ID (except, usually, the native VLAN — see Section 7), so the receiving switch knows which VLAN each frame belongs to and can forward it into the correct broadcast domain.

Fig 3.1 — Access ports connect single-VLAN end devices; the trunk link between switches carries all three VLANs, tagged.

3.3 QUICK COMPARISON

ASPECT	ACCESS PORT	TRUNK PORT
VLANs carried	One	Many (typically all, or an allowed list)
Frame tagging	Untagged	Tagged (except native VLAN)
Typical connection	End device (PC, printer, phone)	Switch-to-switch, switch-to-router
Device VLAN-awareness	None required	Required on both ends of the link

// SECTION 4

Worked Example: Tracing a Tagged Frame

Follow a single frame from PC A (VLAN 20, on Switch A) to PC B (VLAN 20, on Switch B), using the topology from Fig 3.1.

1. **PC A sends a normal, untagged frame** out its network cable — as far as PC A is concerned, VLANs don't exist. It's just sending a frame to a destination MAC address.
2. **Switch A receives the frame on an access port configured for VLAN 20.** Internally, the switch now knows this frame belongs to VLAN 20, even though the frame itself carries no tag yet.
3. **Switch A forwards the frame out the trunk port toward Switch B.** Since the trunk carries multiple VLANs, Switch A inserts an 802.1Q tag with VLAN ID 20 before sending it — this is the only point in the journey where the frame is actually tagged.
4. **Switch B receives the tagged frame on its trunk port,** reads the VLAN ID (20), and now knows which broadcast domain to forward it into.
5. **Switch B strips the tag and forwards the now-untagged frame** out the access port connected to PC B, which is also configured for VLAN 20.

HOP	FRAME STATE	WHY
PC A → Switch A	Untagged	End devices never send tagged frames
Switch A → Switch B (trunk)	Tagged, VLAN 20	Trunk link carries multiple VLANs — tag is required to disambiguate
Switch B → PC B	Untagged	Access port strips the tag before delivery to the end device

> **The tag never reaches the end device.** Tagging exists purely for switch-to-switch (and switch-to-router) communication across shared links. PC A and PC B both send and receive perfectly ordinary, untagged Ethernet frames throughout — the VLAN mechanics are entirely invisible to them.

// SECTION 5

Inter-VLAN Routing

VLANs are isolated broadcast domains by design — which means a device in VLAN 10 *cannot* reach a device in VLAN 20 without something routing between them at Layer 3. Switching alone only ever operates within a single VLAN; getting traffic *between* VLANs always requires a router or a Layer-3-capable switch.

5.1 ROUTER-ON-A-STICK

A single router interface, connected to a trunk port on the switch, is split into multiple **subinterfaces** — one per VLAN — each configured with an IP address acting as that VLAN's default gateway. The router receives tagged frames on the physical interface, and its subinterfaces handle routing between the VLANs using only that one physical link. It's a cost-effective classic design, though the single physical link can become a bandwidth bottleneck as VLAN traffic grows.

5.2 LAYER 3 SWITCH (SVI)

Modern networks more commonly use a **Layer 3 switch** with **SVIs (Switched Virtual Interfaces)** — a virtual routed interface configured directly on the switch for each VLAN, with routing happening in switch hardware at wire speed rather than over a single external link. This eliminates the router-on-a-stick bottleneck entirely and is the standard approach in modern enterprise design.

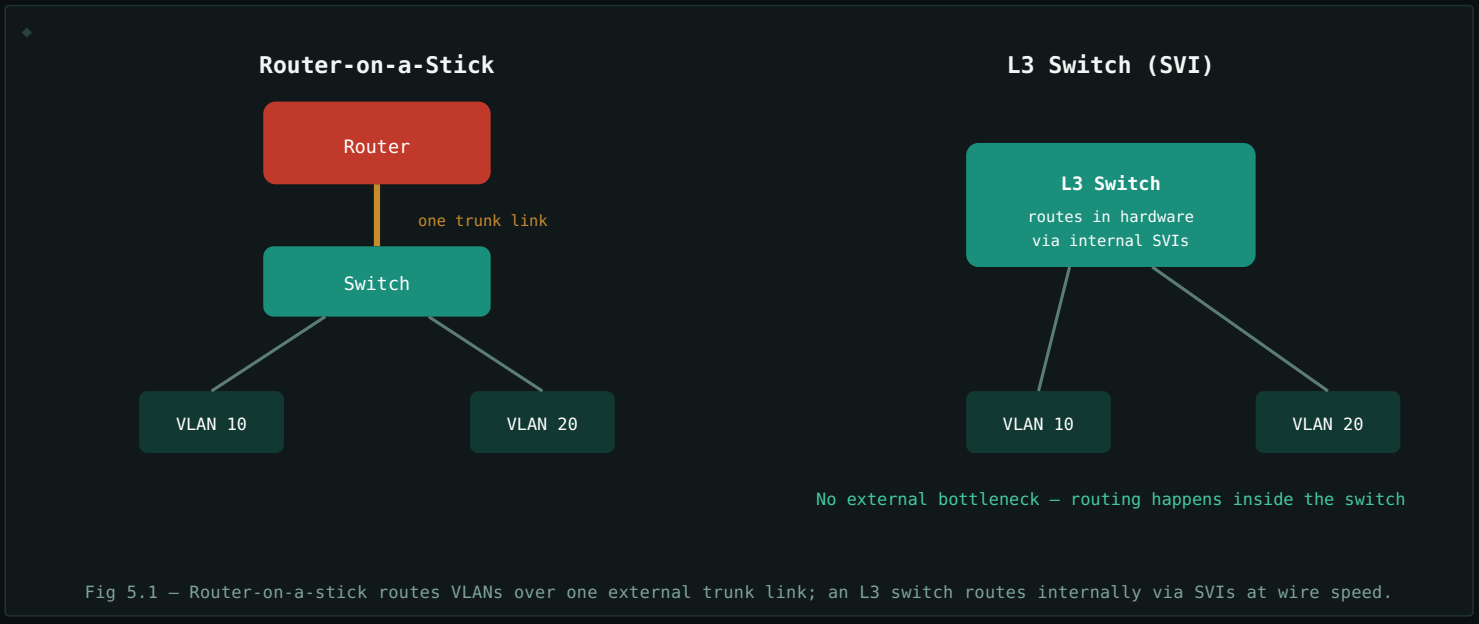


Fig 5.1 — Router-on-a-stick routes VLANs over one external trunk link; an L3 switch routes internally via SVIs at wire speed.

ASPECT	ROUTER-ON-A-STICK	LAYER 3 SWITCH (SVI)
Routing location	External router, over one trunk link	Internal to the switch, in hardware
Bandwidth ceiling	Limited by the single trunk link	Wire-speed, no external bottleneck
Cost	Lower — reuses an existing router	Higher — requires L3-capable switch hardware
Common today	Small networks, labs, legacy setups	Standard in modern enterprise design

// SECTION 6

VLAN Trunking Protocol (VTP)

6.1 THE PROBLEM VTP SOLVES

In a network with dozens of switches, manually creating and keeping the same VLAN list consistent on every single switch is tedious and error-prone — miss one switch, and traffic for a VLAN it doesn't know about simply won't forward correctly. **VTP (VLAN Trunking Protocol)**, a Cisco-originated protocol, lets one switch act as the source of truth for VLAN configuration, automatically propagating VLAN creation, deletion, and renaming to every other switch in the same "VTP domain" over trunk links.

6.2 VTP MODES

MODE	CAN CREATE/EDIT VLANS?	BEHAVIOR
Server	Yes	Originates and propagates VLAN changes to the domain; stores VLAN database in NVRAM.
Client	No	Receives and applies VLAN updates from servers; cannot make local changes.
Transparent	Yes (locally only)	Forwards VTP messages through but keeps its own independent VLAN database, unaffected by the domain.

> **The VTP revision number trap:** VTP uses a revision number to determine which update is "newest." A switch with a higher revision number — even a very old, misconfigured one being added to the network for the first time — can silently **overwrite the entire VLAN database** of a live production network the moment it's plugged in. This has caused real, well-documented outages. Best practice: reset a switch's VTP revision number (e.g., by temporarily switching it to transparent mode) before ever connecting it to a live network.

6.3 WHY MANY NETWORKS AVOID VTP ENTIRELY

Because of the revision-number risk above, and because manually managing VLANs on a modest number of switches isn't actually that burdensome, a great many production networks run every switch in **VTP transparent mode** — getting the trunking benefits of 802.1Q without the single-point-of-failure risk that VTP's automatic propagation introduces.

// SECTION 7

Native VLAN & Common Pitfalls

7.1 WHAT IS THE NATIVE VLAN?

On every trunk port, exactly one VLAN is designated the **native VLAN** — traffic for that VLAN crosses the trunk *untagged*, unlike every other VLAN on the link. This exists mostly for backward compatibility with older, non-802.1Q-aware devices that might be connected to a trunk and need to send/receive at least one VLAN's worth of traffic without understanding tags at all. By default on most platforms, the native VLAN is VLAN 1.

7.2 THE NATIVE VLAN MISMATCH PROBLEM

If the two switches on either end of a trunk link disagree about which VLAN is native — one thinks it's VLAN 1, the other thinks it's VLAN 99 — untagged traffic gets silently misassigned to the wrong VLAN on one end. This is a notoriously easy misconfiguration to make (since it produces no error, just quietly wrong traffic flow) and a classic troubleshooting gotcha in real networks.

> **Best practice:** explicitly configure the native VLAN to match on both ends of every trunk, and many organizations deliberately set the native VLAN to something unused (not VLAN 1) specifically to reduce the security exposure covered in Section 8.

7.3 THE DEFAULT VLAN 1 PROBLEM

VLAN 1 is special on most switch platforms — it can't be deleted, and many control-plane protocols (like the switch's own CDP/LLDP discovery traffic) use it by default. Leaving every unused port in the default VLAN 1, and leaving VLAN 1 as the native VLAN on trunks, means a huge amount of infrastructure ends up sharing one large, default broadcast domain — the opposite of the segmentation VLANs exist to provide. Common hardening advice: move all real traffic off VLAN 1 entirely, and use it only for switch management/control traffic, if at all.

7.4 OTHER COMMON PITFALLS

- **Forgetting to allow a VLAN on a trunk** — by default some platforms restrict which VLANs are permitted across a given trunk; adding a new VLAN to the network but forgetting to update the trunk's allowed list is a very common "why can't these two switches see each other's VLAN" bug.
- **Mismatched trunk encapsulation** — older Cisco gear supported both 802.1Q and a proprietary ISL tagging method; a mismatch here prevents the trunk from forming at all.
- **VLAN sprawl** — creating far more VLANs than the organization actually needs, making the network harder to document, troubleshoot, and secure over time.

// SECTION 8

VLAN Security: Hopping & Hardening

8.1 VLAN HOPPING

VLAN hopping is an attack that lets a device on one VLAN send traffic into a different VLAN it shouldn't have access to — defeating the isolation VLANs are supposed to provide. Two classic techniques:

Switch Spoofing

Many switch ports default to **DTP (Dynamic Trunking Protocol)** auto-negotiation, which can automatically turn a port into a trunk if the connected device asks for it. An attacker's device, pretending to be a switch, can request trunking and — if successful — gain access to every VLAN carried on that trunk, not just its own.

Double Tagging

An attacker crafts a frame with *two* 802.1Q tags stacked. The first switch strips the outer tag (matching the native VLAN) and forwards what's left — which still has the attacker's chosen inner tag — onto the trunk, effectively smuggling the frame into a VLAN the attacker was never authorized to reach. This specific attack only works when the attacker's access VLAN matches the trunk's native VLAN, which is one of several reasons Section 7's native VLAN hardening advice matters.

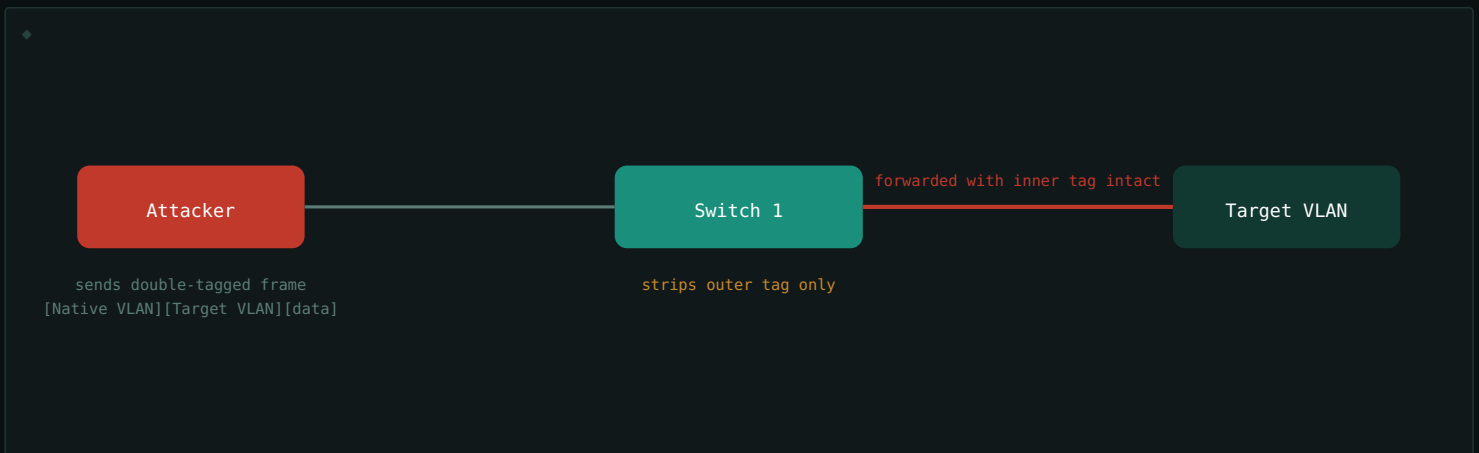


Fig 8.1 — Double-tagging VLAN hopping: the first switch strips only the outer (native VLAN) tag, unintentionally forwarding the frame into the attacker's target VLAN.

8.2 HARDENING CHECKLIST

CONTROL	WHY IT HELPS
Disable DTP auto-negotiation on all ports	Prevents switch spoofing — ports never become trunks unless explicitly configured
Explicitly set access or trunk mode on every port	Removes ambiguity; no port silently becomes something it shouldn't
Set native VLAN to an unused VLAN, not VLAN 1	Breaks the precondition double-tagging attacks rely on
Explicitly prune allowed VLANs on trunks	Limits blast radius even if a trunk is somehow compromised
Disable unused ports and assign them to an unused VLAN	Removes easy physical access points into the network

// SECTION 9

Worked Example: Configuring VLANs

A simplified, vendor-neutral walkthrough of the commands involved in setting up the topology from Fig 3.1 — two VLANs, an access port for each, and a trunk between two switches. Syntax shown is Cisco IOS-style, the de facto reference syntax most platforms' CLIs are modeled after.

9.1 CREATING THE VLANS

```
$ vlan 10
$  name Engineering
$ vlan 20
$  name Accounting
```

9.2 CONFIGURING AN ACCESS PORT

```
$ interface GigabitEthernet0/1
$  switchport mode access
$  switchport access vlan 10
```

This port now belongs exclusively to VLAN 10 — any device plugged in sends and receives untagged frames, which the switch internally treats as VLAN 10 traffic.

9.3 CONFIGURING A TRUNK PORT

```
$ interface GigabitEthernet0/24
$  switchport mode trunk
$  switchport trunk native vlan 999
$  switchport trunk allowed vlan 10,20
```

Note the explicit native VLAN (set to an unused VLAN 999, per the Section 8 hardening advice) and the explicit allowed-VLAN list — both deliberate choices rather than left at insecure defaults.

9.4 VERIFYING THE CONFIGURATION

```
$ show vlan brief
$ show interfaces trunk
```

These commands confirm which ports belong to which VLANs, and which VLANs are actively allowed and tagged across each trunk link — the first place to look when troubleshooting "why can't these two devices talk to each other."

> **In practice:** exact command syntax varies by vendor (Cisco, Juniper, Arista, and others all differ slightly), but the underlying concepts — creating VLANs, assigning access ports, and configuring trunks with an explicit native VLAN and allowed list — are universal across virtually every managed switch platform.

// SECTION 10

Quick Reference & Glossary

TERM	DEFINITION
VLAN	Virtual LAN — a logically isolated broadcast domain within a shared physical switch fabric.
802.1Q	The IEEE standard defining VLAN tagging on Ethernet frames.
VLAN ID	A 12-bit value (1–4094 usable) identifying which VLAN a tagged frame belongs to.
Access Port	A switch port belonging to exactly one VLAN, sending/receiving untagged frames.
Trunk Port	A switch port carrying tagged traffic for multiple VLANs, typically between switches.
Native VLAN	The one VLAN on a trunk whose traffic crosses untagged, for legacy compatibility.
Broadcast Domain	The set of devices that receive each other's broadcast traffic; each VLAN is its own.
Inter-VLAN Routing	Routing traffic between different VLANs, required since switching alone can't cross VLANs.
Router-on-a-Stick	A single router interface with VLAN subinterfaces, routing over one trunk link.
SVI	Switched Virtual Interface — a routed interface for a VLAN configured directly on an L3 switch.
VTP	VLAN Trunking Protocol — propagates VLAN database changes across switches automatically.
DTP	Dynamic Trunking Protocol — auto-negotiates whether a port becomes a trunk; a common attack surface.
VLAN Hopping	An attack technique that lets traffic cross from one VLAN into another it shouldn't reach.
Double Tagging	A VLAN hopping technique using two stacked 802.1Q tags to smuggle a frame past a switch.

End of guide. VLANs are deceptively simple in concept — split one switch into many logical networks — but the details around tagging, trunking, and the native VLAN are where most real-world misconfigurations and security gaps show up. Understanding the tag's journey (Section 4) is the single most useful mental model for troubleshooting almost any VLAN issue.